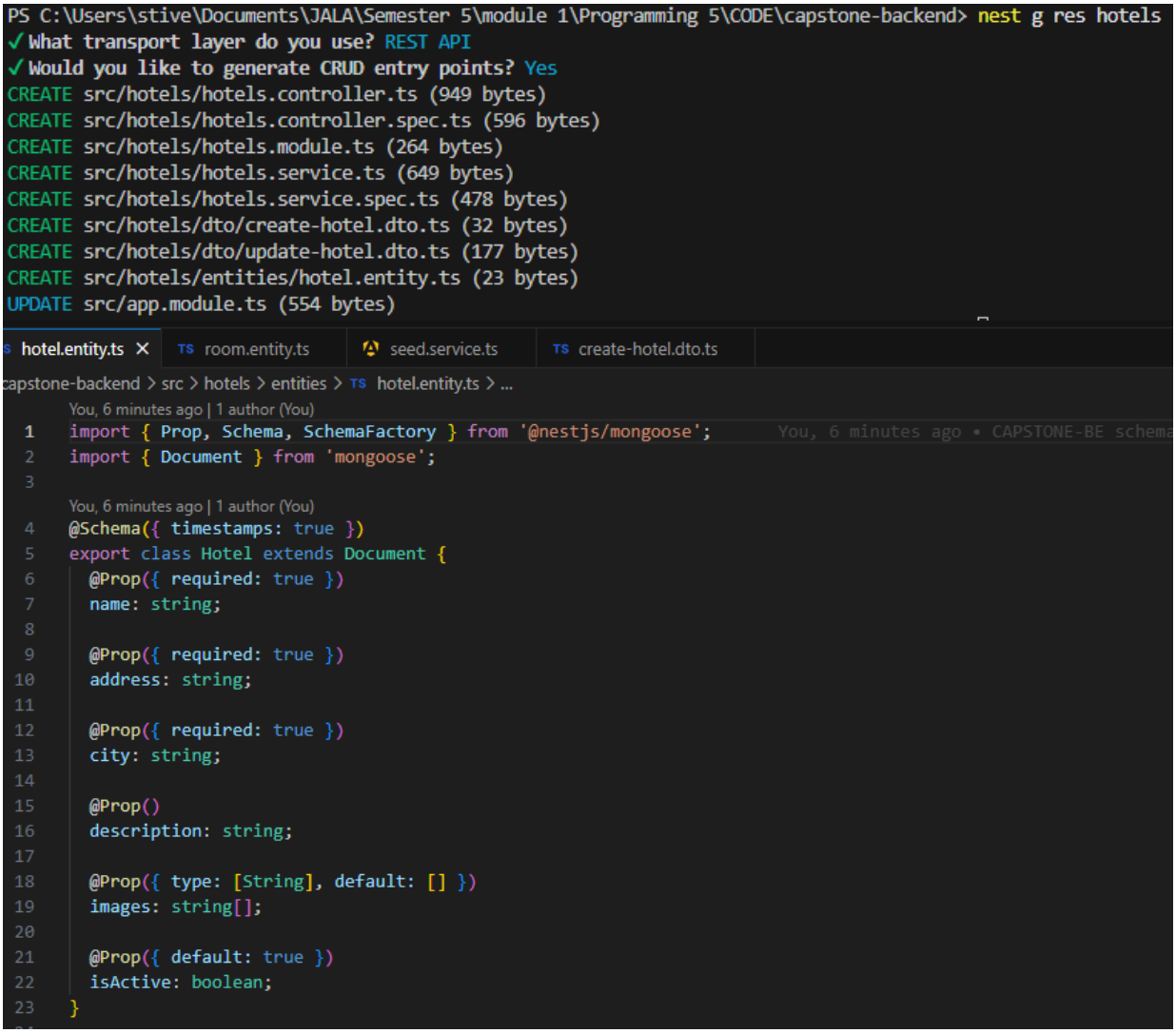
Activity 3

Made by Cristian Galeano

Models

- Create schemas for:
  - Hotel Schema

    Before creating the schema, we first create the base structure for hotels (This will be the same for Room)



- Room Schema

```
PS C:\Users\stive\Documents\JALA\Semester 5\module 1\Programming 5\CODE\capstone-backend> nest g res rooms
✓ What transport layer do you use? REST API
✓ Would you like to generate CRUD entry points? Yes
CREATE src/rooms/rooms.controller.ts (928 bytes)
CREATE src/rooms/rooms.controller.spec.ts (586 bytes)
CREATE src/rooms/rooms.module.ts (257 bytes)
CREATE src/rooms/rooms.service.ts (635 bytes)
CREATE src/rooms/rooms.service.spec.ts (471 bytes)
CREATE src/rooms/dto/create-room.dto.ts (31 bytes)
CREATE src/rooms/dto/update-room.dto.ts (173 bytes)
CREATE src/rooms/entities/room.entity.ts (22 bytes)
UPDATE src/app.module.ts (624 bytes)
PS C:\Users\stive\Documents\JALA\Semester 5\module 1\Programming 5\CODE\capstone-backend>
```

The Enum I use for the schema is based on the rubric that requests the Enum based on the capstone pdf, where it is single for one person, single for two people, single for three people, suite for a couple, and family suite which is the couple and an additional bed for the child as requested in the requirements.

hotel.entity.ts | room.entity.ts M | seed.service.ts M | seed.controller.ts | hotels.service.spec.ts | create-hotel.dto.ts

apstone-backend > src > rooms > entities > room.entity.ts > ...

```ts
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { Document, Types } from 'mongoose';
import { Hotel } from '../../hotels/entities/hotel.entity';

export enum RoomType {
  SIMPLE_SINGLE = 'SIMPLE_SINGLE',
  SIMPLE_DOUBLE = 'SIMPLE_DOUBLE',
  SIMPLE_TRIPLE = 'SIMPLE_TRIPLE',
  SUITE_DOUBLE = 'SUITE_DOUBLE',
  SUITE_FAMILY = 'SUITE_FAMILY',
}

You, 42 seconds ago | 1 author (You)
@Schema({ timestamps: true })
export class Room extends Document {
  @Prop({ type: Types.ObjectId, ref: 'Hotel', required: true })
  hotel: Hotel | Types.ObjectId;

  @Prop({ required: true })
  roomNumber: string;

  @Prop({ type: String, enum: RoomType, default: RoomType.SIMPLE_SINGLE })
  type: RoomType;

  @Prop({ required: true })
  price: number;

  @Prop({ required: true })
  capacity: number;

  @Prop({ default: true })
  isAvailable: boolean;
}

export const RoomSchema = SchemaFactory.createForClass(Room);
Ctrl+I for Command, Ctrl+L for Agent
```

Seed / Data Population

- Create a script or endpoint (POST /seed) that populates the database with at least 3 Hotels and 10 rooms of different types for testing.

```ts
export class SeedService {

  async runSeed() {
    await this.roomModel.deleteMany({});
    await this.hotelModel.deleteMany({});

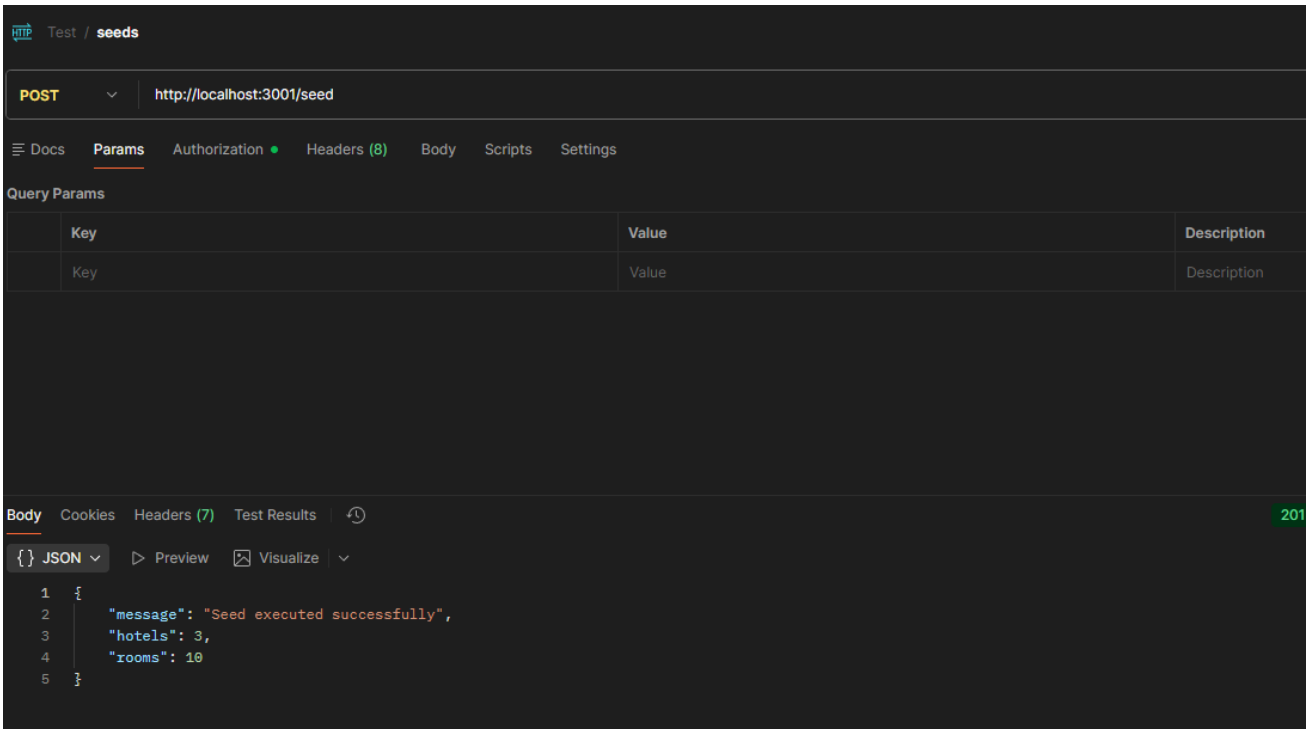
    const hotels = await this.hotelModel.insertMany([
      {
        name: 'Grand Riviera Hotel',
        address: 'Av. Coastal 123',
        city: 'Cancun',
        description: 'Luxury hotel with ocean view',
      },
      {
        name: 'Mountain Peak Lodge',
        address: 'Snowy Peak 456',
        city: 'Aspen',
        description: 'Cozy lodge in the mountains',
      },
      {
        name: 'City Center Inn',
        address: 'Main St 789',
        city: 'New York',
        description: 'Conveniently located in the heart of the city',
      },
    ]);

    const rooms = [       You, 16 minutes ago • CAPSTONE-BE schemas,basic CRUD and seed for hot…
      { hotel: hotels[0]._id, roomNumber: '101', type: RoomType.SIMPLE_SINGLE, price: 100, capacity: 1, isAvailable: true },
      { hotel: hotels[0]._id, roomNumber: '102', type: RoomType.SIMPLE_DOUBLE, price: 150, capacity: 2, isAvailable: true },
      { hotel: hotels[0]._id, roomNumber: '103', type: RoomType.SIMPLE_TRIPLE, price: 200, capacity: 3, isAvailable: true },
      { hotel: hotels[0]._id, roomNumber: '104', type: RoomType.SUITE_DOUBLE, price: 300, capacity: 2, isAvailable: true },
      { hotel: hotels[1]._id, roomNumber: '201', type: RoomType.SIMPLE_SINGLE, price: 110, capacity: 1, isAvailable: true },
      { hotel: hotels[1]._id, roomNumber: '202', type: RoomType.SIMPLE_DOUBLE, price: 160, capacity: 2, isAvailable: true },
      { hotel: hotels[1]._id, roomNumber: '203', type: RoomType.SUITE_FAMILY, price: 350, capacity: 3, isAvailable: true },
      { hotel: hotels[2]._id, roomNumber: '301', type: RoomType.SIMPLE_SINGLE, price: 90, capacity: 1, isAvailable: true },
      { hotel: hotels[2]._id, roomNumber: '302', type: RoomType.SIMPLE_DOUBLE, price: 140, capacity: 2, isAvailable: true },
      { hotel: hotels[2]._id, roomNumber: '303', type: RoomType.SUITE_FAMILY, price: 330, capacity: 3, isAvailable: true },
    ];

    await this.roomModel.insertMany(rooms);

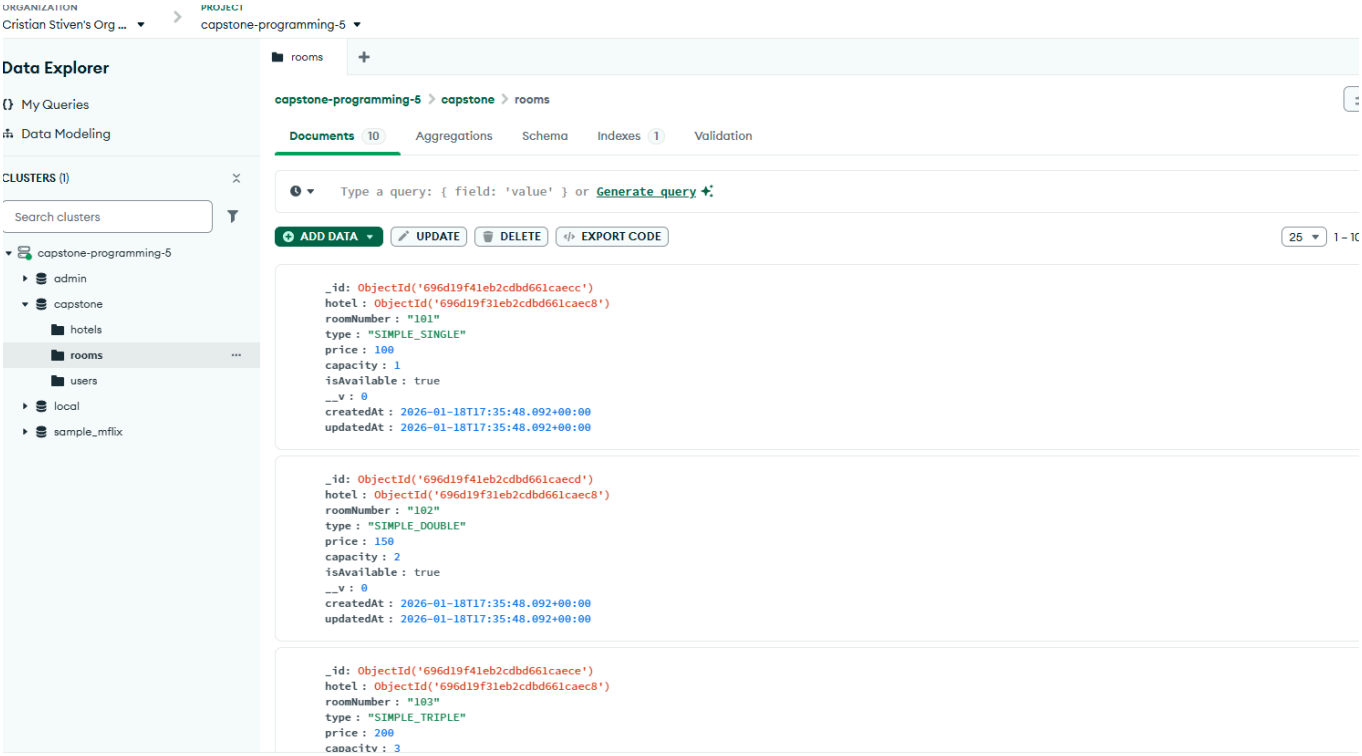
    return { message: 'Seed executed successfully', hotels: hotels.length, rooms: rooms.length };
  }
}
```

- Database populated with consistent data for Week 3 testing.

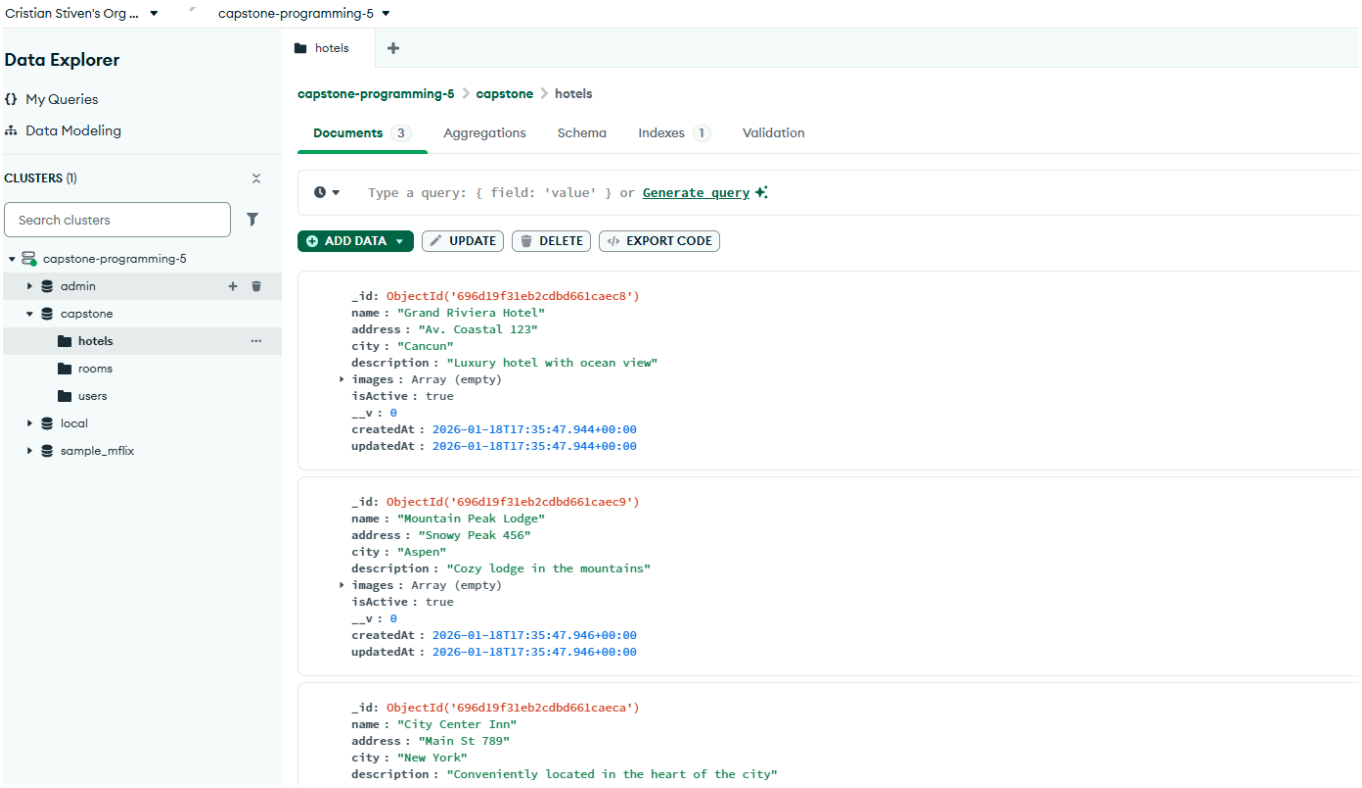


Evidence in Atlas

Rooms



Hotels



MR https://gitlab.com/jala-university1/cohort-3/ES.CSPR-351.GA.T1.26.M1/SB/c.galeano/-/merge_requests/3